```
# Ensemble Learning and Performance Evaluation

# 1. Imported Required Libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.ensemble import (
    StackingClassifier,
    AdaBoostClassifier,
    VotingClassifier
)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)


# 2. Load Dataset

data = load_breast_cancer()

X = data.data
y = data.target

print("Dataset Loaded Successfully!")
print("Dataset Shape:", X.shape)
print("Target Classes:", data.target_names)


# 3. Split Dataset into Training and Testing Sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
60
61
62  # 4. Create Base Classifiers
63
64  lr = Pipeline([
65      ('scaler', StandardScaler()),
66      ('model', LogisticRegression(max_iter=5000))
67  ])
68
69  dt = DecisionTreeClassifier(
70      max_depth=4,
71      random_state=42
72  )
73
74  knn = Pipeline([
75      ('scaler', StandardScaler()),
76      ('model', KNeighborsClassifier(n_neighbors=5))
77  ])
78
79
80  # 5. Store Individual Models
81
82  models = {
83      "Logistic Regression": lr,
84      "Decision Tree": dt,
85      "KNN": knn
86  }
87
88
89  # 6. Implement Stacking Classifier
90
91  stacking = StackingClassifier(
92      estimators=[
93          ('lr', lr),
94          ('dt', dt),
95          ('knn', knn)
96      ],
97      final_estimator=LogisticRegression(max_iter=5000),
98      cv=5
99  )
100
101
102 # 7. Implement AdaBoost Classifier
103
104 adaboost = AdaBoostClassifier(
105     estimator=DecisionTreeClassifier(
106         max_depth=1,
107         random_state=42
108     ),
109     n_estimators=100,
110     learning_rate=0.5,
111     random_state=42
112 )
113
114
115 # 8. Implement Voting Classifier (Soft Voting)
116
117 voting = VotingClassifier(
118     estimators=[
119         ('lr', lr),
120         ('dt', dt),
```

```
        ('knn', knn)
    ],
    voting='soft'
)


# 9. Add Ensemble Models

models["Stacking"] = stacking
models["AdaBoost"] = adaboost
models["Voting"] = voting


# 10. Train Models and Evaluate Performance

results = []

for name, model in models.items():

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    y_prob = model.predict_proba(X_test)[:, 1]

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    auc = roc_auc_score(y_test, y_prob)

    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1,
        "ROC-AUC": auc
    })

    print("\n------------------------------")
    print("MODEL:", name)
    print("------------------------------")

    print(classification_report(
        y_test,
        y_pred,
        target_names=data.target_names
    ))

    # Confusion Matrix

    cm = confusion_matrix(y_test, y_pred)

    plt.figure(figsize=(5, 4))

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        xticklabels=data.target_names
```

```
            yticklabels=data.target_names
        )

        plt.title(name + " - Confusion Matrix")
        plt.xlabel("Predicted")
        plt.ylabel("Actual")
        plt.show()


# 11. Display Performance Comparison Table

results_df = pd.DataFrame(results)

print("\nFINAL PERFORMANCE COMPARISON")

print(results_df.round(4).to_string(index=False))


# 12. Visualize Performance Comparison

results_df.set_index("Model")[
    ["Accuracy", "Precision", "Recall", "F1-Score", "ROC-AUC"]
].plot(
    kind='bar',
    figsize=(12, 6)
)

plt.title("Performance Comparison of ML Models")
plt.xlabel("Machine Learning Models")
plt.ylabel("Performance Score")
plt.ylim(0, 1.05)
plt.xticks(rotation=30)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()


# 13. Identify Best Performing Model

best_model = results_df.loc[
    results_df["F1-Score"].idxmax()
]

print("\nBEST MODEL BASED ON F1-SCORE")
print(best_model.to_string())

print("All Base and Ensemble Models Evaluated.")

```

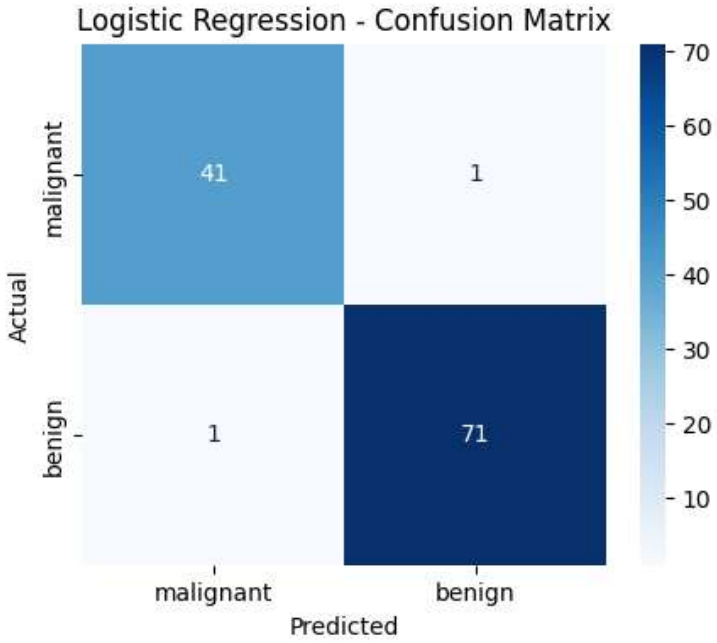
```
Dataset Loaded Successfully!
Dataset Shape: (569, 30)
Target Classes: ['malignant' 'benign']

Training Samples: 455
Testing Samples: 114

------------------------------
MODEL: Logistic Regression
------------------------------
              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```




```
------------------------------
MODEL: Decision Tree
------------------------------
              precision    recall  f1-score   support

   malignant       0.91      0.93      0.92        42
      benign       0.96      0.94      0.95        72

    accuracy                           0.94       114
   macro avg       0.93      0.94      0.93       114
weighted avg       0.94      0.94      0.94       114
```

